BRIGHTMARTDB

Orders & Order_Details

Database Management System — Documentation Report

PROJECT	DETAILS
Database	BrightMartDB
Module	Week 4 – Order Management System
Included	Table design, ER diagram, sample data, customer history, purchase analysis, query outputs

Prepared from the supplied SQL script. Example output tables in this document are illustrative results based on the script's sample order; they are not captured screenshots from a live MySQL session.

1. Introduction

This document describes the Orders and Order_Details module for the BrightMartDB e-commerce database. It records which customer placed an order, when it was placed, the order status and total, and the products and quantities included in each order.

Objectives

• Store order header information and line-item details separately.

• Maintain links to the existing Customer and Product tables.

• Apply primary-key, foreign-key, uniqueness, and value-check constraints.

• Provide queries for order history, product sales, order frequency, and average order value.

2. Orders & Order_Details Table Design

Orders table

Column	Data type	Constraints / purpose
Order_ID	INT	PRIMARY KEY, AUTO_INCREMENT — unique order identifier
Customer_ID	INT	NOT NULL, FOREIGN KEY → Customer(Customer_ID)
Order_Date	DATE	NOT NULL — order date
Total_Amount	DECIMAL(10,2)	NOT NULL, DEFAULT 0, CHECK ≥ 0
Order_Status	VARCHAR(20)	NOT NULL, DEFAULT 'Pending'; allowed: Pending, Shipped, Delivered, Cancelled
(Customer_ID, Order_ID)	UNIQUE	Composite uniqueness constraint as defined in supplied SQL

Order_Details table

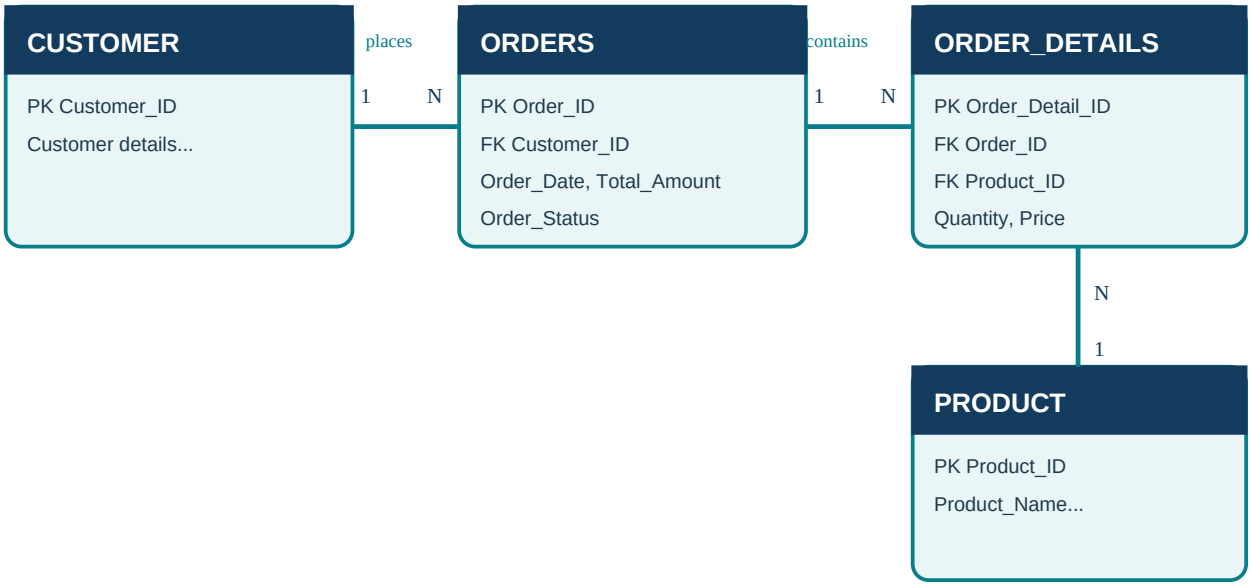
Column	Data type	Constraints / purpose
Order_Detail_ID	INT	PRIMARY KEY, AUTO_INCREMENT
Order_ID	INT	NOT NULL, FOREIGN KEY → Orders(Order_ID), ON DELETE CASCADE
Product_ID	INT	NOT NULL, FOREIGN KEY → Product(Product_ID)
Quantity	INT	NOT NULL, CHECK > 0
Price	DECIMAL(10,2)	NOT NULL, CHECK ≥ 0
(Order_ID, Product_ID)	UNIQUE	A product can appear only once per order; update its quantity instead

Design note

The script references existing Customer and Product tables, so those tables must exist before these statements run. The supplied script uses Product_ID values 101 and 102; those IDs must exist in Product. The total is updated manually after line-item changes, so the application or SQL workflow should recalculate it whenever order details change.

3. Updated ER Diagram

Relationships shown from the supplied foreign keys: one customer can have many orders; one order can have many order-detail rows; one product can appear in many order-detail rows. Order_Details is the line-item table linking orders to products.



Relationship summary

Relationship	Cardinality	Foreign key
Customer → Orders	1 : many	Orders.Customer_ID
Orders → Order_Details	1 : many	Order_Details.Order_ID
Product → Order_Details	1 : many	Order_Details.Product_ID

4. Sample Customer, Order & Order_Details Data

The SQL file inserts one order for Customer_ID 1 using CURDATE(), and two line items for Product_ID 101 and 102. The script later changes the quantity for Product 101 from 2 to 3 and recalculates the order total. The sample below reflects the final state after those statements, assuming the inserts succeed.

Sample Customer (existing parent record; values not included in supplied script)

Customer_ID	Customer_Name	Note
1	Not provided	Customer_ID 1 must already exist in Customer

Orders — final sample state

Order_ID	Customer_ID	Order_Date	Total_Amount	Order_Status
1	1	Current date (CURDATE())	1,800.00	Shipped

Order_Details — final sample state

Order_Detail_ID	Order_ID	Product_ID	Quantity	Price	Line total
1	1	101	3	500.00	1,500.00
2	1	102	1	300.00	300.00

Total check: (3 × ■500) + (1 × ■300) = ■1,800.00.

Data limitation

The supplied SQL does not include Customer or Product insert statements, customer names, or product names. Those values cannot be verified from this file, so they are not invented here. The query outputs below use only the fields and values supported by the script.

5. Customer Order History & Purchase Analysis Report

Customer order history query

The script sorts order records by Customer_ID and then by Order_Date in descending order. With the sample order, the output contains one row: Customer 1, Order 1, current date, ■1,800.00, Shipped.

Customer_ID	Order_ID	Order_Date	Total_Amount	Order_Status
1	1	CURDATE()	■1,800.00	Shipped

Purchase analysis findings

Measure	Illustrative result	Interpretation
Order value	■1,800.00	Updated after Quantity for Product 101 changed to 3
Units ordered	4 units total	3 units of Product 101 + 1 unit of Product 102
Times ordered by product	Product 101: 1; Product 102: 1	Each product appears in one distinct order
Quantity sold by product	Product 101: 3; Product 102: 1	Aggregated quantities from Order_Details
Highest-order customer	Customer 1 (1 order)	Only one order/customer appears in the sample
Average non-cancelled order value	■1,800.00	One non-cancelled order in the sample

Important interpretation

These results are illustrative for the single sample order only. Actual report values depend on the existing Customer and Product records and all data already in BrightMartDB. The script deletes orders marked Cancelled before running the reporting SELECT statements.

6. Query Output Screenshots / Outputs

The uploaded SQL file contains SELECT queries but no actual MySQL Workbench screenshots. Therefore, the following are formatted sample output tables reconstructed from the script, not screenshots captured from a database client.

Output 1 — Order line items with customer and product

Order_ID	Customer_ID	Order_Date	Product_ID	Product_Name	Quantity	Price	Total_Amount	Order_Status
1	1	CURDATE()	101	Depends on Product table	3	■500.00	■1,800.00	Shipped
1	1	CURDATE()	102	Depends on Product table	1	■300.00	■1,800.00	Shipped

Output 2 — Customer order history

Customer_ID	Order_ID	Order_Date	Total_Amount	Order_Status
1	1	CURDATE()	■1,800.00	Shipped

Output 3 — Product order detail

Product_ID	Product_Name	Order_ID	Quantity	Price
101	Depends on Product table	1	3	■500.00
102	Depends on Product table	1	1	■300.00

Output 4 — Product order frequency and total quantity

Product_ID	Times_Ordered	Total_Quantity_Sold
101	1	3
102	1	1

Output 5 — Average order value (excluding cancelled orders)

Average_Order_Value
■1,800.00

To obtain genuine screenshots, run the SQL script in MySQL Workbench (with the parent Customer and Product rows present), execute each SELECT statement, and capture the Result Grid. Replace these illustrative tables with those screenshots if your instructor specifically requires screen captures.

7. SQL Queries Included

Query purpose	What it reports
Order details with customer/product	Order, customer, product, quantity, price, total, and status
Customer order history	Orders sorted by customer and newest date first
Product order details	Products with matching order line items
Product order frequency	Number of distinct orders per product
Total quantity sold	Sum of quantity per product
Customer with most orders	Customer(s) whose order count equals the maximum
Average order value	Average Total_Amount excluding Cancelled orders

8. Conclusion

The Orders and Order_Details module separates order-level information from product line items, enforces key relationships and basic validation, and supports customer-history and purchase-analysis reports. For reliable totals, recalculate Orders.Total_Amount after any Order_Details change; in a production system, this can be automated with application logic or database triggers.

Source: supplied file “dbms task 4(1).sql”.